Phenix AI Agent

2 documentation pages
Generated from local Phenix documentation

Contents

AI automation of structure determination

ai_agent.html

Authors

- Tom Terwilliger, Billy Poon, Pavel Afonine, Dorothee Liebschner, Peter Zwart, Airlie McCoy

Purpose

The Phenix AI Agent automates macromolecular structure determination. You give it your experimental data and (optionally) some guidance, and it figures out which PHENIX programs to run, in what order, with what settings — and then runs them for you automatically.

Think of it as an experienced crystallographer sitting next to you at the computer. It looks at your data, decides what to do first, checks the results, and decides what to do next. If something goes wrong, it tries a different approach. When it's done, it tells you what it found and where to look.

Quickest way to try it: run a tutorial

The fastest way to see the AI Agent in action is to run one of the built-in PHENIX tutorials:

- In the PHENIX GUI, go to File → Tutorials
- Pick a tutorial (e.g. "p9-sad" for Se-SAD phasing, or any MR tutorial with diffraction data)
- Open the AI Agent from the main program list
- The GUI will detect the tutorial automatically — you'll see a blue banner with the tutorial name

All you need to do is click Run. The agent reads the tutorial's README file, extracts the experiment parameters (wavelength, atom type, resolution, etc.), selects a plan template, and runs the entire workflow automatically. No advice, no settings changes, no file selection needed.

The p9-sad tutorial, for example, completes in 5 cycles with 0 failures, producing a model with R-free 0.247 at 1.74 Å resolution.

How the AI Agent works

The AI Agent expects to be given data files from crystallography (mtz files) or from cryo-EM (mrc or ccp4 maps), a sequence file, and optional model files or ligand files. You can tell it what files to use by specifying a directory containing the files or by loading the files in the AI Agent GUI window.

The AI Agent also accepts instructions. You can say things like "first run xtriage and then molecular replacement", or "solve the structure", or "use main.number_of_macro_cycles=1 in phenix refine", or whatever else you want to tell it to do with your X-ray or cryo-EM data.

Note on instructions: if you are supplying PHIL keywords for the agent, specify them exactly as you would on the command line. For example:

```
use main.number_of_macro_cycles=1 in refinement
```

but not:

```
use one macro_cycle in refinement
```

(macro_cycles happens to be a special case that the agent recognizes, but most parameters are not converted like this.)

You can supply instructions in the GUI. If your directory contains a README file the agent can read it and follow the instructions in it.

Structure solution pathways. The AI Agent selects from 17 plan templates covering X-ray MR, SAD, MAD, cryo-EM, ligand fitting, and other workflows. A typical pathway for X-ray data might be:

```
xtriage → autosol → autobuild → refine → molprobity
```

if the data have anomalous signal, or:

```
xtriage → predict_and_build → refine
```

if they do not. For cryo-EM data:

```
mtriage → resolve_cryo_em → dock_in_map → real_space_refine
```

If you supply a ligand, the pathway will end with ligand fitting and model refinement. You can modify most aspects of the pathway with your instructions, as long as the necessary data is available.

How decisions are made. Each cycle, the agent runs a six-node decision pipeline: PERCEIVE (extract metrics, categorize files) → THINK (analyze logs with crystallographic expertise) → PLAN (select next program) → BUILD (construct the command) → VALIDATE (check workflow rules) → OUTPUT (update session). The LLM participates in two nodes (THINK and PLAN); the other four are deterministic. The LLM sets the intent; deterministic code enforces the accuracy.

In Expert mode (the default), the agent also creates a multi-stage strategy plan at the start of the session. After each cycle, a deterministic gate compares metrics to the stage's success criteria and decides whether to advance, retreat, or stop.

What the LLM does not control. The LLM cannot write arbitrary command-line strings. It sets strategy flags (e.g. atom_type=Se, resolution=2.5) which the BUILD node expands through programs.yaml into validated PHIL parameters. File paths, resolution values, R-free flags, and output prefixes are all injected by deterministic code. Any parameter not in the program's allowlist is stripped by the command sanitizer.

What the agent shows you while it runs

Once you click Run, the PHENIX GUI switches to the Agent Progress tab. Here is what you'll see, illustrated with real output from the p9-sad tutorial.

The plan. In Expert mode, the first thing you see is the strategy plan:

```
================================================
 STRATEGY PLAN: SAD/MAD experimental phasing (X-ray)
================================================
 ■ Stage 1: Analyze data quality and anomalous signal
 Goal: Data quality analysis complete
 ■ Stage 2: SAD/MAD phasing and initial model building
 Goal: Experimental phasing complete
 ■ Stage 3: Rebuild and refine model
 Goal: R-free &lt;0.30
 ■ Stage 4: Final refinement with ordered solvent
 Goal: R-free &lt;0.25
 ■ Stage 5: Final model validation
================================================
```

Each stage has a goal. The symbols show status: ■ pending, ● active, ✓ complete, ■ skipped, ✗ failed.

Cycle output. Each program the agent runs appears as a numbered cycle with its decision, reasoning, and result. Here is cycle 2 (autosol) from the same run:

```
Cycle 2: phenix.autosol

Decision: phenix.autosol
 Reasoning: User requested SeMet SAD with additional
 sulfur search using AutoSol, truncating to 2.5 Å,
 wavelength 0.9792 Å, and ~5 Se sites. Xtriage
 indicates usable anomalous signal, so experimental
 phasing is appropriate.
 Source: llm (openai)

File Selection:
 Data_Mtz: p9.sca (Reason: llm_selected)
 Sequence: seq.dat (Reason: llm_selected)

Command:
 phenix.autosol autosol.data=p9.sca seq_file=seq.dat
 autosol.lambda=0.9792 resolution=2.5
 autosol.atom_type=Se mad_ha_add_list=S
 autosol.sites=5 nproc=4

Running: phenix.autosol ... [OK]

[GATE] Phase complete: experimental_phasing →
 build_and_refine
```

The Reasoning field shows exactly why the agent made this choice. The Source field tells you whether the LLM or the rules engine made the decision. The GATE line shows the plan advancing to the next stage.

Stage transitions. When the agent completes or retreats from a stage:

```
✓ STAGE COMPLETE: Analyze data quality
 All steps completed (1/1 cycles) — advancing

■ RETREAT: initial_refinement → molecular_replacement
 R-free stuck above 0.45 after 3 cycles
```

A retreat is not a failure — it means the agent recognized that its current approach isn't working and is trying something different.

What success looks like

Here is the complete output from the p9-sad tutorial (5 cycles, 0 failures):

```
Cycle 1: phenix.xtriage → Data quality OK
 Resolution: 1.74 Å, Space group: I 4
 Anomalous measurability: 0.198

Cycle 2: phenix.autosol → SAD phasing succeeds

Cycle 3: phenix.autobuild → R-free: 0.230

Cycle 4: phenix.molprobity → Clashscore: 12.46

Cycle 5: STOP
 R-free = 0.2466 (&lt; 0.25 target). Stopping.

Final Quality:
 R-free 0.2466 Good
 R-work 0.2295
 Clashscore 12.5 Acceptable
 Rama Outliers 2.4%
 Rotamer Outliers 1.9%
```

```
Key Output Files:
 overall_best_final_refine_001.pdb (model)
 overall_best_refine_data.mtz (data)
 overall_best_refine_map_coeffs.mtz (map coefficients)
```

Understanding the metrics

If R-free is going down cycle by cycle, things are working.

Output files

All output files are in the agent's working directory (usually ai_agent_directory/ inside your project folder).

Your refined model: The .pdb file from the last successful refinement cycle. The Results panel in the GUI shows you exactly where each output file is.

Map coefficients: The .mtz file from the last refinement, containing 2Fo-Fc and Fo-Fc maps. Open this in Coot alongside the refined model to inspect the electron density.

HTML structure report: structure_report.html — a self-contained report with final metrics, an R-free trajectory chart, a workflow timeline, and output file locations. Click the Open Structure Report button in the Results tab to view it.

Text structure report: structure_determination_report.txt — a human-readable summary of the entire determination.

Session summary: session_summary.json — a machine-readable summary with final metrics, stage outcomes, and metric trajectory. Useful for comparing multiple runs programmatically.

What to do with the results: the Coot checklist

The agent produces a model, but you are the scientist. Before depositing or publishing, always inspect the model manually:

- Inspect the difference density map. Display the Fo-Fc map (green/red). Large positive peaks (green) may indicate missing atoms, alternate conformations, or unmodeled ligands.
- Check Ramachandran outliers. In Coot, go to Validate → Ramachandran Plot. Decide whether the density supports each unusual backbone conformation.
- Verify ligand placement. If the agent fit a ligand, examine it in the density. If the real-space correlation (RSCC) is below 0.7, inspect closely.
- Look at B-factor distribution. Regions with very high B-factors may be poorly ordered or incorrectly built.
- Check crystal contacts. Use Coot's symmetry display to verify sensible crystal contacts and correct handling of special positions.

When the AI Agent cannot resolve a problem

If the AI Agent encounters a fatal error (such as a crystal symmetry mismatch between your files, or a missing SHELX installation), it stops the run and produces an HTML diagnosis page that describes the problem and what you can do to fix it. The diagnosis contains three sections: "What went wrong," "Most likely cause," and "How to fix it."

If the agent is unable to produce an LLM-generated diagnosis (e.g. in rules-only mode), a simpler deterministic diagnosis is produced instead.

Common problems and what to do

"R-free stuck above 0.40." The model isn't fitting the data. Check the xtriage output for warnings about space group or twinning. Try a different search model or let the agent use AlphaFold (provide only the sequence).

"Trying a different approach" (RETREAT). The agent recognized that its current strategy isn't working and is backtracking. This is expected behavior — not a failure.

"No matching plan template." The agent couldn't find a strategy for your combination of files and advice. It will still run in reactive mode (choosing programs one at a time). Check that you've provided the right files for your experiment type.

"Safety Stop" or "Agent stopped." Something is fundamentally wrong that the agent cannot fix by trying different programs. The stop message will explain the issue. Check the xtriage output, verify your input files, and look at the failure diagnosis if one was produced.

Settings reference

Analysis Depth (thinking_level). The most important setting:

For most runs, Expert (the default) is the best choice.

Max cycles. Maximum programs to run (default: 20). A typical structure determination takes 5–15 cycles.

Restart mode. Fresh (start new) or Resume (continue from where the previous run left off).

Provider. Which AI to use: Google (default), OpenAI, or Ollama (local).

Verbosity. Quiet (errors only), Normal (decisions and metrics), or Verbose (full detail including file selection).

Job control with the AI Agent

If you have already run an AI Agent job, you can restore it in the GUI by clicking on Job History and then clicking on the run. At the bottom of the Configure panel, you can set:

- Restart mode: Fresh (start a new job) or Resume (keep going)
- Display Session and stop: None, Basic, or Detailed — shows what the agent has done so far without running anything
- Remove last N cycles and stop: Enter a number to roll back that many cycles (e.g. 5 to remove the last 5 decisions)

You can change the advice when you resume. The agent will try to follow your new instructions.

Limitations of the AI Agent

- The agent processes program logs and numerical metrics. It does not see density maps directly — spatial information (ligand shape, disorder) that an experienced crystallographer would see at a glance is not available to the agent.
- One dataset per session. No multi-crystal merging, serial crystallography, or ensemble strategies.
- 23 PHENIX programs are currently registered. Some programs (ensemble_refinement, local sharpening) are not yet available.
- The LLMs used in choosing programs have varying capabilities and can sometimes suggest steps that might not be the best choices.

- AI tools can make mistakes. Use the agent as a helper; don't expect it to always be right. Always inspect the model in Coot.

Privacy

If you use the AI Agent, your log files are sent to the Phenix server, and from there to OpenAI or Google Gemini. That means the data could potentially be used by those providers in any way that they use other data sent to them.

Running locally with Ollama keeps all data on your machine. See below for setup instructions.

Note that access to OpenAI and Gemini through Phenix is non-commercial only.

Running with the Phenix server

Normally you will use Google (Gemini) as the LLM provider. This requires an API key for Google (supplied with Phenix). You can also use Ollama (run on the Phenix server) or OpenAI (also uses a supplied API key).

A shared set of keys is supplied, so you can run without getting your own key. The number of analyses with shared keys is limited, however.

Running locally

You can run on your own machine (not using the Phenix server) by unchecking "Run on server" in the GUI or setting run_on_server=False on the command line.

Your options when running locally:

- Ollama — fully local, requires your own Ollama installation
- Google — your machine calls Google's API (needs your API key)
- OpenAI — your machine calls OpenAI's API (needs your API key)

Running locally with Ollama

If you install Ollama on your machine (typically with a GPU), you can run everything locally without sending any data outside your machine.

After installing Ollama, set up the required models:

```
ollama pull llama3.1:70b
ollama pull llama3.1:8b
ollama pull nomic-embed-text
```

Set the environment variables:

```
setenv CUDA_VISIBLE_DEVICES 0,1,2
setenv OLLAMA_NUM_PARALLEL 6
setenv OLLAMA_HOST 0.0.0.0:11434
```

Then run the agent with:

```
run_on_server=False
provider=ollama
```

Note: the required models may change with Phenix versions.

Running with an API key

If you have a Google API key (set GOOGLE_API_KEY) or an OpenAI API key (set OPENAI_API_KEY), you can run locally with:

```
run_on_server=False
provider=google (or provider=openai)
```

You can test your API keys with:

```
phenix.python $PHENIX/modules/cctbx_project/libtbx/langchain/tests/test_api_keys.py
```

(Note: the path to this test could change.)

Setting up a Google API key

Getting a Google API key requires several steps:

- Set up a Google account: Go to gmail.com and create an account.
- Activate billing: Log in, go to https://console.cloud.google.com/billing and click Activate. You need a credit card but it won't be charged unless you convert to a paid account.
- Create an API key: Go to https://console.cloud.google.com/apis/credentials and create a new credential. Copy the API key.
- Enable the API: Go to https://console.cloud.google.com/apis/library, search for "Generative Language API", select it, and click ENABLE.
- Restrict the key: Go back to your credentials, click your key. Under "Application Restrictions" select "IP Addresses" and enter your IP addresses. Under "API restrictions" select "Restrict key" and choose "Generative Language API".
- Set the environment variable: setenv GOOGLE_API_KEY xxxxxxxx
- In the GUI, uncheck "Run on server" and set provider=google.

Google gives you credits good for 90 days. Beyond that you pay for access.

Setting up an OpenAI API key

Setting up an OpenAI key is simpler but there is no free trial:

- Log into openai.com (or create an account).
- In the left menu, click "API keys".
- Click "+ Create new secret key". Give it a name.
- Copy the key immediately (it's shown only once). Store it securely.
- setenv OPENAI_API_KEY sk-xxxxxxxx
- In the GUI, uncheck "Run on server" and set provider=openai.
- You pay for access from the start.

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command lineai_analysislog_file = None log_as_simple_string = None file_list_as_simple_string = None display_text_as_simple_string = None summary_as_simple_string = None analysis_as_simple_string = None load_existing_analysis = True program_name = None analysis_file_name = None summary_file_name = None write_files = True timeout = 180 display_results = True log_directory = None job_id = None history_file = None history_simple_string =

None max_history = 10 iterate_agent = True max_cycles = 20 restart_mode = *fresh resume Fresh: start a new analysis from scratch. Resume: continue from the previous run's session.auto_recovery = True Automatically attempt to recover from certain structured errors (e.g., ambiguous data labels in MTZ files). Set to False for debugging or when you want full manual control.project_advice = None input_directory = None Directory containing input files and optional README with instructions. If a README file is found, its contents will be used as additional advice for the agent.preprocess_advice = True Use LLM to preprocess and clarify user advice into structured format. This helps the agent better understand informal or brief instructions.readme_file_patterns = README README.txt README.dat README.md notes.txt Filenames to look for when extracting advice from input_directory. Search is case-insensitive.max_readme_chars = 5000 Truncate README files longer than this to avoid excessive LLM input.maximum_automation = True use_rules_only = False When True, the agent skips ALL LLM calls: - Program selection uses deterministic rules from YAML config - Directive extraction uses simple pattern matching - Advice preprocessing returns raw advice unchanged - Session summaries are skipped This runs completely offline without external API calls. Useful for testing or when API quota is exhausted.use_llm = True When False, equivalent to use_rules_only=True. The agent runs without any LLM calls, using only deterministic workflow rules for program selection. Set to False for fully offline operation.thinking_level = none basic advanced *expert Controls the depth of expert crystallographer reasoning applied at each decision point. none: No expert reasoning — original agent behavior. basic: Adds an LLM reasoning call with log analysis and strategy memory tracking. advanced (default): Full pipeline including structural validation, file metadata tracking, and expert knowledge base lookup. Increases LLM cost per cycle but substantially improves decision quality. expert: Adds goal-directed planning with multi-stage strategy, gate evaluation, retreat logic, hypothesis testing, and session reports. Requires advanced mode features (structural validation, structure model) as prerequisites.abort_on_red_flags = True When True, the agent will stop if it detects critical issues like experiment type changes or impossible workflow states. Set to False to continue despite red flags (not recommended).abort_on_warnings = False When True, the agent will also stop on warning-level issues like R-free spikes or missing resolution. Default is False (warnings are logged but don't stop execution).verbosity = quiet *normal verbose Controls how much detail is shown in the agent output. quiet: Only errors and final result. normal: Key decisions, metrics, and reasoning (default). verbose: Full detail including file selection and internal state.dry_run = False For testing: substitutes pre-recorded logs and outputs instead of running actual PHENIX programs. Use with use_rules_only=True for fully offline testing.dry_run_scenario = "xray_basic" Which test scenario to use for dry_run mode. Scenario folders are in tests/scenarios/.summarize_only = False Skip running any cycles and just generate a summary of the existing session. Useful for re-generating summaries or testing the summary functionality.include_llm_assessment = True When True, the final session summary will include an LLM-generated assessment...

- job_title = None Job title in PHENIX GUI, not used on command line
- ai_analysislog_file = None log_as_simple_string = None file_list_as_simple_string = None display_text_as_simple_string = None summary_as_simple_string = None analysis_as_simple_string = None load_existing_analysis = True program_name = None analysis_file_name = None summary_file_name = None write_files = True timeout = 180 display_results = True log_directory = None job_id = None history_file = None history_simple_string = None max_history = 10 iterate_agent = True max_cycles = 20 restart_mode = *fresh resume Fresh: start a new analysis from scratch. Resume: continue from the previous run's session.auto_recovery = True Automatically attempt to recover from certain structured errors (e.g., ambiguous data labels in MTZ files). Set to False for debugging or when you want full manual control.project_advice = None input_directory = None Directory containing input files and optional README with instructions. If a README file is found, its contents will be used as additional advice for the agent.preprocess_advice = True Use LLM to preprocess and clarify user advice into structured format. This helps the agent better understand informal or brief

instructions.readme_file_patterns = README README.txt README.dat README.md notes.txt Filenames to look for when extracting advice from input_directory. Search is case-insensitive.max_readme_chars = 5000 Truncate README files longer than this to avoid excessive LLM input.maximum_automation = True use_rules_only = False When True, the agent skips ALL LLM calls: - Program selection uses deterministic rules from YAML config - Directive extraction uses simple pattern matching - Advice preprocessing returns raw advice unchanged - Session summaries are skipped This runs completely offline without external API calls. Useful for testing or when API quota is exhausted.use_llm = True When False, equivalent to use_rules_only=True. The agent runs without any LLM calls, using only deterministic workflow rules for program selection. Set to False for fully offline operation.thinking_level = none basic advanced *expert Controls the depth of expert crystallographer reasoning applied at each decision point. none: No expert reasoning — original agent behavior. basic: Adds an LLM reasoning call with log analysis and strategy memory tracking. advanced (default): Full pipeline including structural validation, file metadata tracking, and expert knowledge base lookup. Increases LLM cost per cycle but substantially improves decision quality. expert: Adds goal-directed planning with multi-stage strategy, gate evaluation, retreat logic, hypothesis testing, and session reports. Requires advanced mode features (structural validation, structure model) as prerequisites.abort_on_red_flags = True When True, the agent will stop if it detects critical issues like experiment type changes or impossible workflow states. Set to False to continue despite red flags (not recommended).abort_on_warnings = False When True, the agent will also stop on warning-level issues like R-free spikes or missing resolution. Default is False (warnings are logged but don't stop execution).verbosity = quiet *normal verbose Controls how much detail is shown in the agent output. quiet: Only errors and final result. normal: Key decisions, metrics, and reasoning (default). verbose: Full detail including file selection and internal state.dry_run = False For testing: substitutes pre-recorded logs and outputs instead of running actual PHENIX programs. Use with use_rules_only=True for fully offline testing.dry_run_scenario = "xray_basic" Which test scenario to use for dry_run mode. Scenario folders are in tests/scenarios/.summarize_only = False Skip running any cycles and just generate a summary of the existing session. Useful for re-generating summaries or testing the summary functionality.include_llm_assessment = True When True, the final session summary will include an LLM-generated assessment of the workflow. When False, only the structured summary is gener...

- log_file = None
- log_as_simple_string = None
- file_list_as_simple_string = None
- display_text_as_simple_string = None
- summary_as_simple_string = None
- analysis_as_simple_string = None
- load_existing_analysis = True
- program_name = None
- analysis_file_name = None
- summary_file_name = None
- write_files = True
- timeout = 180
- display_results = True
- log_directory = None
- job_id = None

• history_file = None

• history_simple_string = None

• max_history = 10

• iterate_agent = True

• max_cycles = 20

• restart_mode = *fresh resume Fresh: start a new analysis from scratch. Resume: continue from the previous run's session.

• auto_recovery = True Automatically attempt to recover from certain structured errors (e.g., ambiguous data labels in MTZ files). Set to False for debugging or when you want full manual control.

• project_advice = None

• input_directory = None Directory containing input files and optional README with instructions. If a README file is found, its contents will be used as additional advice for the agent.

• preprocess_advice = True Use LLM to preprocess and clarify user advice into structured format. This helps the agent better understand informal or brief instructions.

• readme_file_patterns = README README.txt README.dat README.md notes.txt Filenames to look for when extracting advice from input_directory. Search is case-insensitive.

• max_readme_chars = 5000 Truncate README files longer than this to avoid excessive LLM input.

• maximum_automation = True

• use_rules_only = False When True, the agent skips ALL LLM calls: - Program selection uses deterministic rules from YAML config - Directive extraction uses simple pattern matching - Advice preprocessing returns raw advice unchanged - Session summaries are skipped This runs completely offline without external API calls. Useful for testing or when API quota is exhausted.

• use_llm = True When False, equivalent to use_rules_only=True. The agent runs without any LLM calls, using only deterministic workflow rules for program selection. Set to False for fully offline operation.

• thinking_level = none basic advanced *expert Controls the depth of expert crystallographer reasoning applied at each decision point. none: No expert reasoning — original agent behavior. basic: Adds an LLM reasoning call with log analysis and strategy memory tracking. advanced (default): Full pipeline including structural validation, file metadata tracking, and expert knowledge base lookup. Increases LLM cost per cycle but substantially improves decision quality. expert: Adds goal-directed planning with multi-stage strategy, gate evaluation, retreat logic, hypothesis testing, and session reports. Requires advanced mode features (structural validation, structure model) as prerequisites.

• abort_on_red_flags = True When True, the agent will stop if it detects critical issues like experiment type changes or impossible workflow states. Set to False to continue despite red flags (not recommended).

• abort_on_warnings = False When True, the agent will also stop on warning-level issues like R-free spikes or missing resolution. Default is False (warnings are logged but don't stop execution).

• verbosity = quiet *normal verbose Controls how much detail is shown in the agent output. quiet: Only errors and final result. normal: Key decisions, metrics, and reasoning (default). verbose: Full detail including file selection and internal state.

• dry_run = False For testing: substitutes pre-recorded logs and outputs instead of running actual PHENIX programs. Use with use_rules_only=True for fully offline testing.

• dry_run_scenario = "xray_basic" Which test scenario to use for dry_run mode. Scenario folders are in tests/scenarios/.

• summarize_only = False Skip running any cycles and just generate a summary of the existing session. Useful for re-generating summaries or testing the summary functionality.

• include_llm_assessment = True When True, the final session summary will include an LLM-generated assessment of the workflow. When False, only the structured summary is generated (faster, no API call needed).

• analysis_mode = *standard agent_session advice_preprocessing Type of analysis: standard (single log file), agent_session (multi-step AI agent run with structured summary), or advice_preprocessing (process user advice into structured format)

• session_json = None For agent_session mode, the session data as JSON string

• raw_advice = None For advice_preprocessing mode, the combined raw advice string

• experiment_type_hint = None For advice_preprocessing mode, experiment type (xray/cryoem)

• file_list_hint = None For advice_preprocessing mode, comma-separated input file names

• user_advice_for_directives = None For directive_extraction mode, the processed user advice string

• original_files = None

• project_state_json = None

• resolution = None

• api_version = None When set, indicates request is in JSON format (v2.0)

• request_json = None Full JSON request containing files, history, settings, etc. Used when api_version is set.

• gui_mode = False When True, run sub-jobs via the program's native PHENIX launcher (template_launcher/run_program) for proper .pkl and .eff output, and send callbacks to the GUI for sub-job registration and live window opening. Falls back to easy_run if native launcher is not available. Set automatically by the GUI launcher.

• auto_open_sub_jobs = True

• display_and_stop = *None basic detailed Show the current session history and exit without running any cycles. basic: compact summary (program, result, R-free per cycle). detailed: full detail including files, metrics, and reasoning.

• remove_last_n = None Remove the last N cycles from the session and exit. Useful to roll back failed or unwanted cycles before resuming.

• communicationrun_on_server = True Run job on Phenix serverwait_for_server = False If server is busy or down, wait up to max_wait_timeupdate_wait_time_if_down = 5 Time to wait before trying again on server if it is downupdate_wait_time_if_busy = 5 Time to wait before trying again on server if it is busymax_wait_time = 450 Maximum wait timemax_server_tries = 1 Maximum calls to serverstop_if_internet_not_available = True Stop if attempting to predict models online and internet is not availablecycle = 1 Marker that can be used to identify cycleverbose = False Verbose output on communicationsprovider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.

• run_on_server = True Run job on Phenix server

• wait_for_server = False If server is busy or down, wait up to max_wait_time

• update_wait_time_if_down = 5 Time to wait before trying again on server if it is down

• update_wait_time_if_busy = 5 Time to wait before trying again on server if it is busy

• max_wait_time = 450 Maximum wait time

• max_server_tries = 1 Maximum calls to server

• stop_if_internet_not_available = True Stop if attempting to predict models online and internet is not available

• cycle = 1 Marker that can be used to identify cycle

• verbose = False Verbose output on communications

• provider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.

• rest_serverurl = None The URL for the Phenix REST server Normally set automaticallyurl_type = prediction *ai Type of server url (prediction or ai). Normally set automaticallyport = None The port for interacting with the Phenix REST server Normally set automaticallytoken = None Authentication token for accessing the Phenix REST server. Normally set automaticallytimeout = 5 Time to keep trying to connect to a serverquick_check_interval = 1 Time in seconds between job status checkscheck_interval = 5 Time in seconds between job status checksmax_tries = 20 Number of tries to get result from servermax_tries_on_availability = 2 Number of tries to see if server is upstop_if_server_not_available = True Stop if server is not available (wrong url or down)job_size = *small medium large Size of job (small, medium, large)requires_gpu = False Job requires GPUrunning_server_test = False This is a test jobverbose = False Verbose output

• url = None The URL for the Phenix REST server Normally set automatically

• url_type = prediction *ai Type of server url (prediction or ai). Normally set automatically

• port = None The port for interacting with the Phenix REST server Normally set automatically

• token = None Authentication token for accessing the Phenix REST server. Normally set automatically

• timeout = 5 Time to keep trying to connect to a server

• quick_check_interval = 1 Time in seconds between job status checks

• check_interval = 5 Time in seconds between job status checks

• max_tries = 20 Number of tries to get result from server

• max_tries_on_availability = 2 Number of tries to see if server is up

• stop_if_server_not_available = True Stop if server is not available (wrong url or down)

• job_size = *small medium large Size of job (small, medium, large)

• requires_gpu = False Job requires GPU

• running_server_test = False This is a test job

• verbose = False Verbose output

• guiGUI-specific parameter required for output directoryoutput_dir = None

• output_dir = None

AI analysis of Phenix results

ai_analysis.html

Authors

- Nigel Moriarty, Peter Zwart, Thomas C Terwilliger

Purpose

The Phenix AI analysis tool is designed to help you interpret your output files from a run in the Phenix GUI. It analyzes your log file and any output written to the GUI and the names of any output files that are supplied to the GUI and creates a summary of the run. Then it analyzes this summary in the context of the Phenix documentation to describe how this run fits into the framework of structure determination and suggests next steps. The AI analysis tool can be accessed in the Phenix GUI in the Results section of most Phenix GUI runs, next to the Log file button. You can also access it under the Tools menu.

Google and OpenAI API Keys

You can use Ollama (run on the Phenix server) to run the AI analysis, or you can use Google (gemini) or OpenAI. If you use Google or OpenAI, API keys for these servers are used. You can run AI analyses with Google or OpenAI without getting your own key, as a shared set of keys is supplied. The number of analyses with these shared keys is limited, however.

How the AI analysis works

The AI analysis is carried out in two steps. In the first step, the log file, along with text written to the GUI and names of output files supplied to the GUI are summarized using a Langchain analysis with reranking using FlashRank. In the second step, the summary from the first step is analyzed in the context of all the Phenix documentation, transcripts of Phenix tutorial videos, and Phenix publications to provide background and to suggest next steps to take.

This type of AI does not save or learn from your questions. However the information in your log file is sent to the Phenix server, and from there, on to Google gemini.

What to do with the AI analysis

The purpose of the AI analysis is to help you interpret your output and to suggest ideas for next steps. You should keep in mind that these tools can make mistakes and give you incorrect interpretations and poor suggestions at times, so you want to just take the information as suggestions to think about.

You can combine this AI analysis with the Phenix chatbot . The chatbot can give you interactive answers to your questions using the same database of information as the AI analysis. This allows you to follow up on the AI analysis with questions to the chatbot. You can also paste part of the output from the AI analysis into the chatbot along with a question to get more context.

Limitations of the AI analysis

The AI analysis is limited to the sources that is supplied with, so it only knows about Phenix, and it only knows what is in the documentation, the videos and newsletters, and the papers we have supplied.

AI tools like this one can also just make mistakes and give incorrect answers. This does not seem to happen too often with this tool, but you need to always be on alert when using it. Use the tool as a helper, don't expect it to always be right.

If a detail is missing in the documentation, the AI analysis will not know it.

Privacy in the AI analysis

If you use the AI analysis tool, your log files are sent to the Phenix server, and from there on to OpenAI and Gemini. That means the data could potentially be used by OpenAI, and Google in any way that they use other AI data that is sent to them.

Note that access to OpenAI and Gemini is non-commerical only.

List of all available keywords

job_title = None Job title in PHENIX GUI, not used on command lineai_analysislog_file = None log_as_simple_string = None file_list_as_simple_string = None display_text_as_simple_string = None summary_as_simple_string = None analysis_as_simple_string = None load_existing_analysis = True program_name = None analysis_file_name = None summary_file_name = None write_files = True timeout = 180 display_results = True analysis_mode = *standard agent_session advice_preprocessing directive_extraction failure_diagnosis Type of analysis: standard (single log file), agent_session (multi-step AI agent run with structured summary), advice_preprocessing (process user advice into structured format), directive_extraction (extract structured directives from advice), or failure_diagnosis (LLM diagnosis of a diagnosable-terminal error)include_llm_assessment = True For agent_session mode, whether to include LLM-generated assessmentsession_json = None For agent_session mode, the session data as JSON stringraw_advice = None For advice_preprocessing mode, the combined raw advice stringexperiment_type_hint = None For advice_preprocessing mode, experiment type (xray/cryoem)file_list_hint = None For advice_preprocessing mode, comma-separated input file namesuser_advice_for_directives = None For directive_extraction mode, the processed user advice stringfailure_error_type = None For failure_diagnosis mode: error type key from diagnosable_errors.yamlfailure_error_text = None For failure_diagnosis mode: encoded error excerpt from the failing programfailure_program = None For failure_diagnosis mode: the PHENIX program that failedfailure_log_tail = None For failure_diagnosis mode: encoded last section of the failing program logcommunicationrun_on_server = True Run job on Phenix serverwait_for_server = False If server is busy or down, wait up to max_wait_timeupdate_wait_time_if_down = 5 Time to wait before trying again on server if it is downupdate_wait_time_if_busy = 5 Time to wait before trying again on server if it is busymax_wait_time = 450 Maximum wait timemax_server_tries = 1 Maximum calls to serverstop_if_internet_not_available = True Stop if attempting to predict models online and internet is not availableverbose = False Verbose output on communicationsprovider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.rest_serverurl = None The URL for the Phenix REST server Normally set automaticallyurl_type = prediction *ai Type of server url (prediction or ai). Normally set automaticallyport = None The port for interacting with the Phenix REST server Normally set automaticallytoken = None Authentication token for accessing the Phenix REST server. Normally set automaticallytimeout = 5 Time to keep trying to connect to a serverquick_check_interval = 1 Time in seconds between job status checkscheck_interval = 5 Time in seconds between job status checksmax_tries = 20 Number of tries to get result from servermax_tries_on_availability = 2 Number of tries to see if server is upstop_if_server_not_available = True Stop if server is not available (wrong url or down)job_size = *small medium large Size of job (small, medium, large)requires_gpu = False Job requires GPUrunning_server_test = False This is a test jobverbose = False Verbose outputguiGUI-specific parameter required for output

directoryoutput_dir = None

- job_title = None Job title in PHENIX GUI, not used on command line
- ai_analysislog_file = None log_as_simple_string = None file_list_as_simple_string = None display_text_as_simple_string = None summary_as_simple_string = None analysis_as_simple_string = None load_existing_analysis = True program_name = None analysis_file_name = None summary_file_name = None write_files = True timeout = 180 display_results = True analysis_mode = *standard agent_session advice_preprocessing directive_extraction failure_diagnosis Type of analysis: standard (single log file), agent_session (multi-step AI agent run with structured summary), advice_preprocessing (process user advice into structured format), directive_extraction (extract structured directives from advice), or failure_diagnosis (LLM diagnosis of a diagnosable-terminal error)include_llm_assessment = True For agent_session mode, whether to include LLM-generated assessmentsession_json = None For agent_session mode, the session data as JSON stringraw_advice = None For advice_preprocessing mode, the combined raw advice stringexperiment_type_hint = None For advice_preprocessing mode, experiment type (xray/cryoem)file_list_hint = None For advice_preprocessing mode, comma-separated input file namesuser_advice_for_directives = None For directive_extraction mode, the processed user advice stringfailure_error_type = None For failure_diagnosis mode: error type key from diagnosable_errors.yamlfailure_error_text = None For failure_diagnosis mode: encoded error excerpt from the failing programfailure_program = None For failure_diagnosis mode: the PHENIX program that failedfailure_log_tail = None For failure_diagnosis mode: encoded last section of the failing program log
- log_file = None
- log_as_simple_string = None
- file_list_as_simple_string = None
- display_text_as_simple_string = None
- summary_as_simple_string = None
- analysis_as_simple_string = None
- load_existing_analysis = True
- program_name = None
- analysis_file_name = None
- summary_file_name = None
- write_files = True
- timeout = 180
- display_results = True
- analysis_mode = *standard agent_session advice_preprocessing directive_extraction failure_diagnosis Type of analysis: standard (single log file), agent_session (multi-step AI agent run with structured summary), advice_preprocessing (process user advice into structured format), directive_extraction (extract structured directives from advice), or failure_diagnosis (LLM diagnosis of a diagnosable-terminal error)
- include_llm_assessment = True For agent_session mode, whether to include LLM-generated assessment
- session_json = None For agent_session mode, the session data as JSON string
- raw_advice = None For advice_preprocessing mode, the combined raw advice string

• experiment_type_hint = None For advice_preprocessing mode, experiment type (xray/cryoem)

• file_list_hint = None For advice_preprocessing mode, comma-separated input file names

• user_advice_for_directives = None For directive_extraction mode, the processed user advice string

• failure_error_type = None For failure_diagnosis mode: error type key from diagnosable_errors.yaml

• failure_error_text = None For failure_diagnosis mode: encoded error excerpt from the failing program

• failure_program = None For failure_diagnosis mode: the PHENIX program that failed

• failure_log_tail = None For failure_diagnosis mode: encoded last section of the failing program log

• communicationrun_on_server = True Run job on Phenix serverwait_for_server = False If server is busy or down, wait up to max_wait_timeupdate_wait_time_if_down = 5 Time to wait before trying again on server if it is downupdate_wait_time_if_busy = 5 Time to wait before trying again on server if it is busymax_wait_time = 450 Maximum wait timemax_server_tries = 1 Maximum calls to serverstop_if_internet_not_available = True Stop if attempting to predict models online and internet is not availableverbose = False Verbose output on communicationsprovider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.

• run_on_server = True Run job on Phenix server

• wait_for_server = False If server is busy or down, wait up to max_wait_time

• update_wait_time_if_down = 5 Time to wait before trying again on server if it is down

• update_wait_time_if_busy = 5 Time to wait before trying again on server if it is busy

• max_wait_time = 450 Maximum wait time

• max_server_tries = 1 Maximum calls to server

• stop_if_internet_not_available = True Stop if attempting to predict models online and internet is not available

• verbose = False Verbose output on communications

• provider = ollama *google openai Provider for AI analysis. Ollama is cheapest, google is quickest, OpenAI is most thorough.

• rest_serverurl = None The URL for the Phenix REST server Normally set automaticallyurl_type = prediction *ai Type of server url (prediction or ai). Normally set automaticallyport = None The port for interacting with the Phenix REST server Normally set automaticallytoken = None Authentication token for accessing the Phenix REST server. Normally set automaticallytimeout = 5 Time to keep trying to connect to a serverquick_check_interval = 1 Time in seconds between job status checkscheck_interval = 5 Time in seconds between job status checksmax_tries = 20 Number of tries to get result from servermax_tries_on_availability = 2 Number of tries to see if server is upstop_if_server_not_available = True Stop if server is not available (wrong url or down)job_size = *small medium large Size of job (small, medium, large)requires_gpu = False Job requires GPUrunning_server_test = False This is a test jobverbose = False Verbose output

• url = None The URL for the Phenix REST server Normally set automatically

• url_type = prediction *ai Type of server url (prediction or ai). Normally set automatically

• port = None The port for interacting with the Phenix REST server Normally set automatically

• token = None Authentication token for accessing the Phenix REST server. Normally set automatically

• timeout = 5 Time to keep trying to connect to a server

• quick_check_interval = 1 Time in seconds between job status checks

- check_interval = 5 Time in seconds between job status checks
- max_tries = 20 Number of tries to get result from server
- max_tries_on_availability = 2 Number of tries to see if server is up
- stop_if_server_not_available = True Stop if server is not available (wrong url or down)
- job_size = *small medium large Size of job (small, medium, large)
- requires_gpu = False Job requires GPU
- running_server_test = False This is a test job
- verbose = False Verbose output
- guiGUI-specific parameter required for output directoryoutput_dir = None
- output_dir = None